

OBJECTIVES

- Understand when to use Redis, PostgreSQL, and Cassandra in a microservices architecture for Enterprise Solutions.
- Integrate Redis as a caching layer into a Spring Boot service.
- Model high-volume write data with Apache Cassandra.
- Apply table partitioning and sharding concepts on a PostgreSQL database.

DISCUSSION

Scenario	Best DB?	Key reason
A user logs in; their session token must be read on every API call (< 1 ms).		
A legal team needs to search documents by category, author, and date range with joins.		
10,000 IoT sensors each emit an event every second; the data is rarely re-read.		
You need to cache the result of an expensive aggregation query for 30 seconds.		
A document's comment count must always be exactly correct — no eventual consistency tolerated.		
You need to store user activity logs; writes dominate 95% of traffic.		

INDIVIDUAL EXERCISES

Task / Exercise C1 [60 Minutes] (Configuring Redis) :

In high-traffic DMS scenarios, repeatedly fetching the same document from PostgreSQL is expensive. In this exercise you will introduce Redis as a caching layer inside the Documents Spring Boot service so that frequently-read documents are served from memory after the first database hit.

Part 1 : Add Redis to the Docker Compose stack

1. Add the following Redis service block to your docker-compose.yml:

```
redis:
  image: redis:7-alpine
  ports:
    - "6379:6379"
  command: redis-server --save 60 1 --loglevel warning
```

2. Start the stack with docker compose up -d and verify Redis is running:

```
docker exec -it <redis_container_id> redis-cli ping
# Expected output: PONG
```

Part 2 : Configure Spring Boot to use Redis

3. Add the following dependency to your Documents service build.gradle (or pom.xml):

```
// build.gradle
implementation 'org.springframework.boot:spring-boot-starter-data-redis'
implementation 'org.springframework.boot:spring-boot-starter-cache'
```

4. In application.yml, configure the Redis connection:

```
spring:
  data:
    redis:
      host: ${REDIS_HOST:localhost}
      port: 6379
  cache:
    type: redis
    redis:
      time-to-live: 60000    # 60 seconds TTL
```

5. Enable caching in your main application class with @EnableCaching.

Part 3 : Annotate the service layer

6. Add @Cacheable to the method that fetches a single document by ID:

```
@Cacheable(value = "documents", key = "#id")
public Document getDocumentById(Long id) {
    return documentRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Not found"));
}
```

7. Add @CacheEvict on the update/delete methods so the cache stays consistent:

```
@CacheEvict(value = "documents", key = "#id")
public void deleteDocument(Long id) { ... }
```

Part 4 : Verify caching behaviour

8. Send two consecutive GET requests for the same document via Postman.
9. Using redis-cli, confirm the cached value exists:

```
redis-cli keys '*'
redis-cli get 'documents::1'
```

10. Delete the document, then confirm the cache entry has been evicted.

Questions :

- What happens to cached data if the Documents service pod restarts?
- What happens to cached data if the Redis pod restarts?
- If two replicas of the Documents service share the same Redis instance, and replica A updates a document – what risk arises if you use @Cacheable without @CacheEvict?

Task / Exercise C2 [20 Minutes] (Cassandra for High-Volume Comment Events)

Comments are written far more often than they are queried as a full list per document, making them a good candidate for Cassandra's write-optimised model. In this exercise you will replace the Comments service's H2/PostgreSQL store with Apache Cassandra.

Part 1 : Start a Cassandra instance

```
# Add to docker-compose.yml
cassandra:
  image: cassandra:4.1
  ports:
    - "9042:9042"
  environment:
    CASSANDRA_CLUSTER_NAME: dms-cluster
    CASSANDRA_DC: datacenter1
    HEAP_NEWSIZE: 128M
    MAX_HEAP_SIZE: 512M
  healthcheck:
    test: ["CMD", "cqlsh", "-e", "describe keyspaces"]
    interval: 30s
    retries: 10
```

Part 2 : Create the keyspace and table via CQL

Wait for Cassandra to start (check healthcheck), then run:

```
docker exec -it <cassandra_container> cqlsh

CREATE KEYSPACE IF NOT EXISTS dms
  WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

USE dms;

CREATE TABLE IF NOT EXISTS comments (
  doc_id      UUID,
  comment_id  TIMEUUID,
  content     TEXT,
  author      TEXT,
  PRIMARY KEY (doc_id, comment_id)
) WITH CLUSTERING ORDER BY (comment_id DESC);
```

Part 3 : Connect Spring Boot to Cassandra

1. Add the dependency:

```
implementation 'org.springframework.boot:spring-boot-starter-data-cassandra'
```

2. Configure application.yml:

```
spring:
  cassandra:
    keyspace-name: dms
    contact-points: ${CASSANDRA_HOST:localhost}
    port: 9042
    local-datacenter: datacenter1
    schema-action: CREATE_IF_NOT_EXISTS
```

3. Define the entity using Spring Data Cassandra annotations (@Table, @PrimaryKey, @PrimaryKeyColumn).
4. Replace the existing CommentRepository with a CassandraRepository.

Part 4 : Test with Postman

5. POST 10 comments for the same document ID.
6. GET all comments for that document ID and confirm they arrive newest-first.

Discussion: Why is PRIMARY KEY (doc_id, comment_id) designed this way? What would break if doc_id were the only primary key?

Task / Exercise C3 [10 Minutes] (PostgreSQL Partitioning & Sharding Concepts)

As the DMS grows, a single documents table will slow down range queries (e.g. "all documents created this month"). Partitioning divides the table into smaller physical segments while keeping a single logical view. Sharding takes this further by distributing partitions across multiple database nodes.

Part 1 : Range partitioning on created_at

Run the following in your PostgreSQL container (psql):

```
-- 1. Rename the existing documents table
ALTER TABLE documents RENAME TO documents_old;

-- 2. Create a partitioned parent table
CREATE TABLE documents (
  id          BIGSERIAL,
  title       TEXT NOT NULL,
  created_at  DATE NOT NULL DEFAULT CURRENT_DATE,
  owner       TEXT
) PARTITION BY RANGE (created_at);

-- 3. Create child partitions (one per quarter of 2025)
CREATE TABLE documents_2025_q1
  PARTITION OF documents
  FOR VALUES FROM ('2025-01-01') TO ('2025-04-01');

CREATE TABLE documents_2025_q2
```

```

PARTITION OF documents
FOR VALUES FROM ('2025-04-01') TO ('2025-07-01');

CREATE TABLE documents_2025_q3
PARTITION OF documents
FOR VALUES FROM ('2025-07-01') TO ('2025-10-01');

CREATE TABLE documents_2025_q4
PARTITION OF documents
FOR VALUES FROM ('2025-10-01') TO ('2026-01-01');

-- 4. Migrate data
INSERT INTO documents SELECT * FROM documents_old;

```

Part 2 : Query and verify partition pruning

```

-- Confirm rows land in the correct partition
SELECT tableoid::regclass AS partition, *
FROM documents
WHERE created_at BETWEEN '2025-01-01' AND '2025-03-31';

-- Inspect the query plan to confirm partition pruning
EXPLAIN ANALYSE
SELECT * FROM documents
WHERE created_at = '2025-02-15';

```

Expected: the EXPLAIN output should show that only documents_2025_q1 is scanned, not all partitions.

ASSIGNMENT

- Exercise C3 shall be demonstrated by students for the next lab session.
- Run the same K6 load profile (ramp to 100 concurrent users posting comments over 2 minutes) against two configurations:
 - Your original Spring Boot Comments service backed by PostgreSQL
 - The new one backed by Cassandra.
 - Record throughput (req/s), p95 write latency, and error rate for both.
 - Submit: the two K6 summary outputs, a paragraph explaining what the numbers show, and a one-sentence answer to: "At what scale would you expect the gap to widen further, and why?"

OPTIONAL EXERCISES

Task / Exercise P0 – Questions

- The CAP theorem says a distributed system can only guarantee two of: Consistency, Availability, Partition Tolerance. Which two does Redis prioritise? Which two does Cassandra prioritise?
- What is the difference between replication and partitioning? Can you have one without the other?
- A colleague proposes: "Let's just use PostgreSQL for everything – it supports JSON, it's fast enough." How do you respond?

Task / Exercise P1 – Cache Invalidation is Hard

You have two replicas of the Documents service behind the Gateway's load balancer. Both share the same Redis instance. A document is updated through Replica A. Replica B still has the old value in its local memory (assume you also added an in-memory Caffeine layer as an L1 cache in front of Redis).

- Draw the full cache invalidation problem for this two-level scenario.
- Propose and implement a solution. There are multiple valid approaches – justify yours.
- What is a "cache stampede" and under what load condition could it occur in your implementation? How would you protect against it?

Task / Exercise P2 – Break Cassandra on Purpose

Extend your docker-compose.yml to run a 3-node Cassandra cluster. Change replication_factor to 3. Then deliberately break the cluster and document what you observe:

1. Stop one node. Can you still write a comment? Can you still read comments? Why?
2. Stop two nodes. What happens now? Why is the behaviour different?
3. Run nodetool status on the surviving node and interpret every column of the output.
4. Restore all nodes. Describe what Cassandra does to resynchronise the data.

Expected output: a written report (max half a page) describing the fault behaviour you observed, mapped to the CAP theorem properties.

Task / Exercise P3 – Partitioning Gone Wrong

A colleague partitions the users table by the column email (first letter). Identify at least three problems with this design decision and propose a better alternative for a table expected to grow to 100 million users. Justify every choice.

Task / Exercise P4 – Continue from Exercise C3

Part 3 : Hash partitioning (optional extension)

Repeat the exercise using PARTITION BY HASH (id) with 4 partitions. Hash partitioning is preferred when data has no natural range key and you need an even distribution across nodes (shards).

```
CREATE TABLE documents_hash (
  id          BIGSERIAL,
  title       TEXT NOT NULL,
  created_at  DATE DEFAULT CURRENT_DATE
) PARTITION BY HASH (id);

CREATE TABLE documents_hash_0 PARTITION OF documents_hash
  FOR VALUES WITH (MODULUS 4, REMAINDER 0);
CREATE TABLE documents_hash_1 PARTITION OF documents_hash
  FOR VALUES WITH (MODULUS 4, REMAINDER 1);
-- ... repeat for REMAINDER 2 and 3
```

Part 4 : Reflection questions

- What is the difference between partitioning (single node) and sharding (multiple nodes)?
- Which partitioning strategy (range, list, hash) would you choose for a log table indexed by timestamp? Justify your answer.
- If documents_2025_q1 becomes disproportionately large, what options do you have?